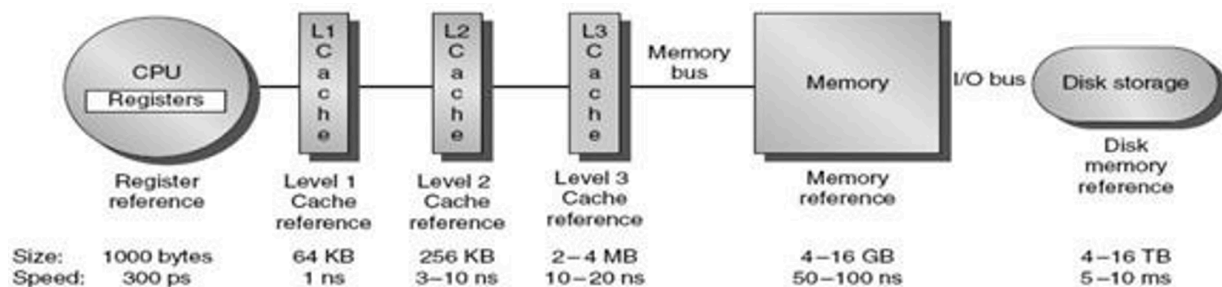


APPENDIX B

What is Cache?

- Cache is the first (fastest) memory the CPU checks when it needs data. It stores frequently used data to speed up processing.
- high-speed buffer between RAM and CPU, storing frequently accessed data, costlier than RAM but cheaper than CPU registers, optimizing processing speed.
- **Cache Hit:** Data is found in cache (fast access).
- **Cache Miss:** Data is not in cache → fetch from slower main memory.
- **Block:** A fixed-size chunk of data transferred between main memory and cache.
- **Line:** A single entry in the cache that stores one block of data along with metadata like the tag.
- **L1, L2, and L3** caches are progressively larger and slower memory levels, where L1 is fastest and closest to the CPU, L2 is intermediate, and L3 is largest and shared among cores.



Why Cache Works: Locality

- **Temporal Locality:** If we use data now, we'll likely use it again soon.
- **Spatial Locality:** If we use one memory location, nearby locations are also likely needed soon.

A **cache miss delay** depends on **latency** (time to get the first word) and **bandwidth** (time to get the remaining block). Hardware handles the miss. In **in-order execution**, the processor **stalls** until data arrives; in **out-of-order execution**, only the instruction needing the data **waits**, while others may **continue**.

Not all program data stays in main memory; in **virtual memory**, some is on disk, divided into fixed-size blocks called pages. A page is either in memory or on disk. If a processor accesses a page not in memory, a **page fault** occurs, moving the whole page from disk to memory. Page faults are slow, handled by software, and the processor usually switches tasks instead of stalling. Like caches, virtual memory relies on locality and shows similar size, speed, and cost patterns between cache vs memory and memory vs disk.

Cache Access Time: time required to access word from cache when hit occurs

Miss Penalty: Extra time to fetch data from lower memory after a miss

Miss Delay: The total time the processor is stalled or slowed down because of a cache miss.

Concept	Formula / Definition
CPU Time	$(\text{CPU Cycles} + \text{Memory Stall Cycles}) \times \text{Clock Cycle Time}$ OR $\text{IC} \times (\text{CPI}_{\text{execution}} + \text{Memory Stall Clock Cycles} / \text{Instruction}) \times \text{Clock Cycle Time}$ OR $(i=1 \sum n \text{IC}_i \times \text{CPI}_i) \times \text{Clock cycle time}$
Memory Stall Cycles	Number of Misses $\times$ Miss Penalty OR $\text{IC} \times (\text{Misses} / \text{Instruction}) \times \text{Miss Penalty}$ OR $\text{IC} \times \text{Miss Rate} \times (\text{Memory Accesses} / \text{Instruction}) \times \text{Miss Penalty}$ OR $(\text{Misses} / \text{Instruction}) \times (\text{Total Miss Latency} - \text{Overlapped Miss Latency})$ OR $\text{IC} \times \text{reads per inst} \times \text{read miss rate} \times \text{read miss penalty} + \text{IC} \times \text{writes per inst} \times \text{writes miss rate} \times \text{writes miss penalty}$
Miss Ratio	Misses / Accesses (i.e hits+miss) OR 1 - Hit Ratio.
Hit Ratio	Hits / Accesses
Average CPU Access Time	$(\text{Hit Ratio} \times \text{Cache Access Time}) + (\text{Miss Ratio} \times \text{Miss Penalty})$ OR Hit Time + Miss Rate $\times$ Miss Penalty OR $\text{Hit Time}_{\text{L1}} + \text{Miss Rate}_{\text{L1}} \times (\text{Hit Time}_{\text{L2}} + \text{Miss Rate}_{\text{L2}} \times \text{Miss Penalty}_{\text{L2}})$
Misses per Instruction	Misses / Instructions (often shown as misses per 1000 instructions)
Misses per Instruction (L1 & L2)	$(\text{Misses}_{\text{L1}} / \text{Instruction}) \times \text{Hit Time}_{\text{L2}} + (\text{Misses}_{\text{L2}} / \text{Instruction}) \times \text{Miss Penalty}_{\text{L2}}$
Cache Index Size	$2^{\text{index}} = \text{Cache Size} / (\text{Block Size} \times \text{Set Associativity})$
Note	Misses per instruction is preferred by CPU architects because it is independent of CPU techniques like speculation.

Cache → CPU: When the CPU reads from the cache, it usually requests just a word (fast, direct).

Memory → Cache: When the CPU has a cache miss, it doesn't just fetch the single word from main memory — it fetches a block (also called a "cache line"), which is a group of words due to locality.

Write Policies: Define how and where data is written during a write operation (cache vs memory).

Write Through

- Data written to both cache block and lower-level memory
- **Pros:** simpler hardware, always up-to-date memory copy, easier data coherency
- **Cons:** slower cache writes, increases memory bus traffic
- Often paired with **no-write allocate**

Write Back

- Data written only to cache block; memory updated only when block is replaced
- Uses **dirty bit** to track modified (dirty) blocks
- **Pros**: faster writes, reduced memory bandwidth usage, lower power consumption
- **Cons**: complex hardware, possible data loss on failure
- Usually paired with **write allocate**

Write Strategies (on Cache Miss): *Define whether the cache allocates space when a write misses in the cache.*

Write Allocate

- Block is brought into cache on a write miss, then written to
- **Pros**: improves future cache hits, exploits temporal locality
- **Cons**: may cause cache pollution if data is rarely reused
- Common with **write-back**

No-Write Allocate

- Write miss bypasses cache; data written directly to lower-level memory
- **Pros**: avoids cache pollution, simpler cache management
- **Cons**: no reuse benefit from written data
- Common with **write-through**

Cache Mapping: method used to store data from main memory into the cache

- **Direct Mapping:** Each memory block maps to exactly one cache line predetermined by formula. Simple and fast but can cause many conflicts if multiple blocks map to the same line (overwrite). Direct mapping's performance is directly proportional to the Hit ratio.
- **Fully Associative Mapping:** Any memory block can be placed in any cache line.
- **Set-Associative Mapping:** Memory block maps to a set, and can go into any line within that set.

Block offset tells which word inside the block.

Line number tells which line in the cache.

Tag identifies which block is currently stored.

Block Replacement: On a capacity miss, the controller must choose a block to replace.

- Random: Pick randomly.
- Least Recently Used (LRU): Replace the block unused for the longest time.
- First In, First Out (FIFO): Replace the oldest block.

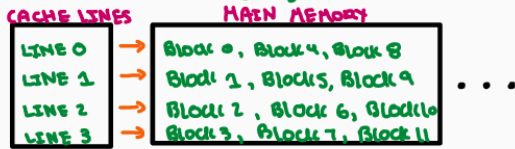
Note: True LRU is costly; often approximated using pseudo-LRU bits.

Note: If blocks are repeating, use LRU; otherwise, use FIFO

NOTES FOR BLOCK MAPPING, BLOCK FINDING (USING TAGS), AND BLOCK REPLACEMENT, SEE EXAMPLE Qs IN BOOK

DIRECT MAPPING

- 1 Block = 4 WORDS, each cache line maps to one specific memory block
- main memory has 128 WORDS, $128/4 = 32$ BLOCKS
- 16 WORDS cache memory, each line holds 1 Block, $16/4 = 4$ lines in cache

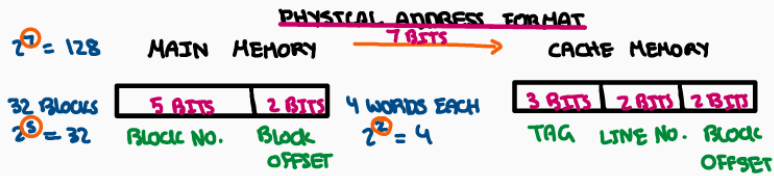


FORMULA FOR MAPPING → (Block No.) MOD (NO. OF CACHE LINES)

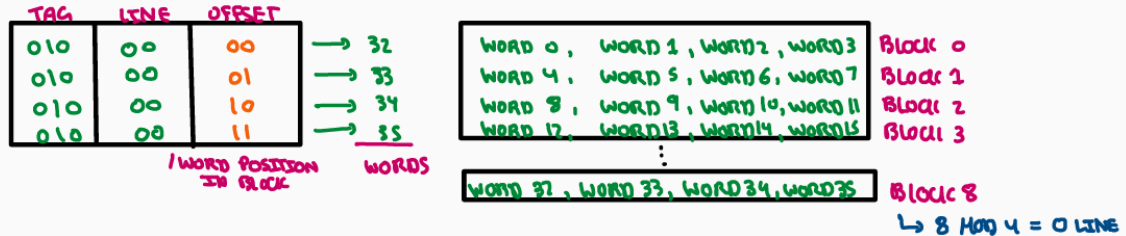
$= 6 \text{ MOD } 4 \rightarrow \text{LINE 2}$

$16 \text{ MOD } 4 \rightarrow \text{LINE 0}$

IN ORDER TO LOCATE WORD IN MAIN MEMORY →

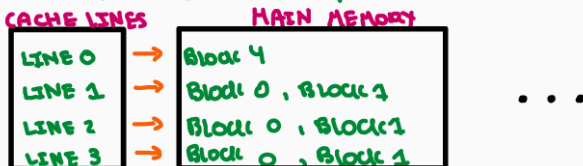


CONSIDER LINE 0 HAS TAG BITS 010 →

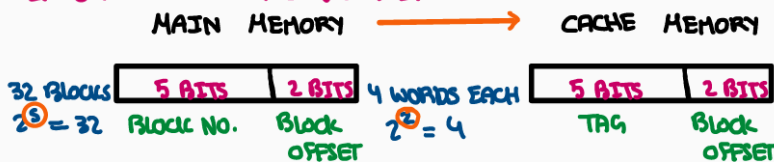


FULLY ASSOCIATIVE

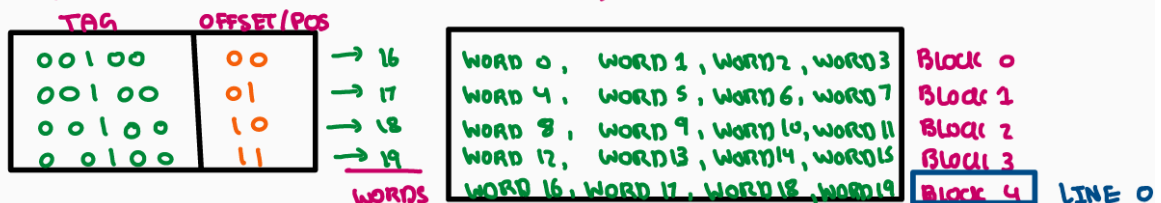
- physical address format same for both MM and cache
- one Block each line, but can be any block (no formula)
- 128 word MM (32 blocks), 16 word cache memory (4 lines)



IN ORDER TO LOCATE WORD IN MM →



CONSIDER LINE 0 HAVING TAG BITS 00100 →



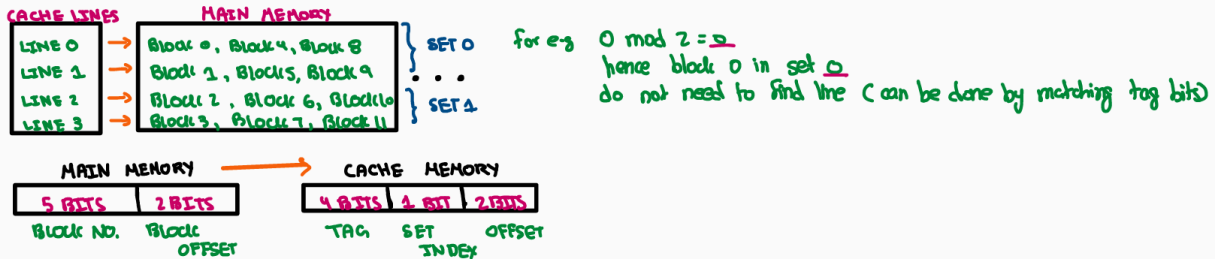
K WAY SET ASSOCIATIVE MAPPING

- a set consists of k lines (given), a block can map to any of the lines in the set
- calculate set value → $\text{SET NO.} = (\text{BLOCK NO.}) \bmod (\text{NUM OF SETS}) \text{ OR } k \bmod n$
- number of sets → $\text{NUM OF SETS} = (\text{NUM OF CACHE LINES}) / k$
- use direct mapping to decide which block arrives in cache
- consider fully associative mapping to decide which block sustains in given line
- physical address formats are changed for MH and CH

$$k = 2, S = 2$$

MH → 128 words, 32 blocks

CH → 16 words, 4 lines



EXAMPLE NUMERICAL MAPPING

M way set associative cache

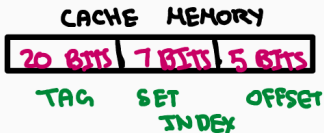
capacity 16 KB

block size 8 words, word length 32 bits

$$M = 4$$

size of physical address space is 4 GB

calc num of tag bits



physical address → $2^{32} = 4 \text{ GB}$ hence 32 bits
 block offset → 8 words of 4 Bytes each (since size given)
 $8 \times 4 \text{ B} = 32 \text{ B}$ size of a block
 $2^5 = 32$ hence 5 bits offset.
 num of sets → $\text{cache lines} / k = 2^9 / 4 = 2^7$ sets
 cache lines → $\text{capacity} / \text{block size} = 16 \text{ KB} / 32 \text{ B} = 2^9$ lines
 tag bits → $32 - 7 - 5 = 20$

EXAMPLE NUMERICAL FULLY ASSOCIATIVE

fully associative, 8 lines

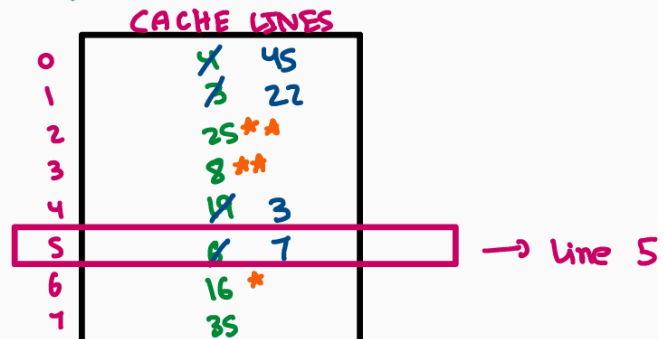
request sequence 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

to which cache line block 7 will accommodate?

OVERWRITE / REPLACEMENT ALGOS

LRU → least recently used

FIFO → first in first out



EXAMPLE NUMERICAL - K WAY

Search Notebook

4 way

sequence $\rightarrow$ 0, 255, 1, 4, 3, 8, 133, 159, 216, 129, 63, 8,
48, 32, 73, 92, 155

cache lines = 16

$$\text{num of sets} = \frac{16}{4} = 4$$

0	Block 0 48	SET 0	USING LRU IF BLOCKS NOT REPEATING $\rightarrow$ LIFO IF BLOCKS REPEATING $\rightarrow$ LRU
1	Block 4 32		
2	Block 8*		
3	Block 216 93		
4	Block 1 155	SET 1	
5	Block 133		
6	Block 129		
7	Block 73		
8	Block 255	SET 2	
9	Block 159		
10			
11			
12	Block 3	SET 3	
13	Block 63		
14			
15			

CACHE OPTIMIZATION

Goal: Improve average memory access time = hit time + (miss rate $\times$ miss penalty)

Categories:

- **Reduce miss rate:** larger block size, larger cache size, higher associativity
- **Reduce miss penalty:** multilevel caches, read priority over writes
- **Reduce hit time:** avoid address translation when indexing the cache

Misses:

- **Compulsory Miss:** Happens the **first time a memory block is accessed**, e.g., accessing $A[0]$ at program start — the cache has never seen it before.
- **Conflict Miss:** Occurs when **multiple blocks map to the same cache set**, e.g., $A[0]$ and $A[64]$ both map to set 0 in a 64-line direct-mapped cache, causing evictions.
- **Capacity Miss:** Happens when the **cache cannot hold all needed blocks**, e.g., looping through a 10KB array with a 4KB cache — blocks keep getting evicted and reloaded.

1. Larger Block Size – Reduces Miss Rate

Pros: grabs more nearby data, fewer first-time misses, better for programs with sequential access

Cons: fewer blocks fit in cache, higher data fetch cost, may kick out useful data in small caches

2. Larger Cache Size – Reduces Capacity Misses

Pros: stores more data, fewer evictions, better for big or long-running programs

Cons: slower access time, higher cost and power use, takes up more space

3. Higher Associativity – Reduces Conflict Misses

Pros: reduces slot fights, more flexible data placement, fewer conflict misses

Cons: slower to search, more complex hardware, slightly longer access time

4. Multilevel Caches – Reduces Miss Penalty

Pros: backup cache helps on misses, reduces delay to main memory, balances speed and size

Cons: adds complexity, uses more power, harder to manage

5. Read Miss Priority Over Writes (use write buffer for dirty blocks) – Reduces Miss Penalty

Pros: keeps CPU running smoothly (reads block CPU progress), reads happen faster, avoids stalling on reads

Cons: write operations may pile up, risk of delay if write buffer gets full

6. Avoid Address Translation (virtual to physical) When Indexing (using part of address - page offset to find cache line/set) – Reduces Hit Time

Pros: faster cache access, skips translation delay, speeds up L1 cache

Cons: tricky to manage, risk of duplicate data (aliasing many virtual to one physical), adds design complexity

DYNAMIC SCHEDULING - SEE SLIDES

Scoreboarding is a technique used in CPUs to allow instructions to run out of order without messing up the results. It was introduced in the CDC 6600.

It works like a control desk that tracks:

- Which units are busy
- Which registers are in use
- What data is ready

The CPU uses this "scoreboard" to make decisions on-the-fly (in real time) during execution. Everything goes through the scoreboard.

The 4 Stages of Scoreboarding:

1. **Issue** (Check for WAW and Structural Hazards):
 - Is the required functional unit free?
 - Is any other instruction already writing to the same destination register?

Operation	Cycles
Load/INT	1
Add/Sub	2
Multiply	10
Divide	40

- If no conflicts, the instruction is issued.
2. **Read Operands** (Check for RAW Hazards):
 - Are the input values ready?
 - Is any earlier instruction going to write to the source registers?
 - If all clear, operands are read.
 3. **Execute**:
 - The instruction performs its operation (like add or multiply).
 - Happens as soon as it can, not necessarily in the program's order.
 4. **Write Result** (Check for WAR Hazards):
 - Can we safely write the result to the register?
 - Wait if any earlier instruction hasn't yet read from that register.
 - Write only when it's safe.

SCOREBOARD EXAMPLE

Instruction status:

Instruction

J

k

Read

Exec

Write

Issue

Oper

Comp

Result

LD

F6

34+ R2

LD

F2

45+ R3

MULTD

F0

F2

F4

SUBD

F8

F6

F2

DIVD

F10

F0

F6

ADD

F6

F8

F2

Functional unit status:

dest

S1

S2

FU

FU

Fj?

Fk?

Busy

Op

Fi

Fj

Fk

Qj

Qk

Rj

Rk

Time Name

Integer

Mult1

Mult2

Add

Divide

No

No

No

No

No

Register result status:

F0

F2

F4

F6

F8

F10

F12

...

F30

Clock

FU

Key Features:

- Allows out-of-order execution and out-of-order completion
- Instructions are still issued in order
- Handles RAW, WAR, and WAW hazards
- The scoreboard ensures that all decisions are made safely during execution

Components:

F_i, F_j, F_k - Destination and source registers of the instruction

Q_j, Q_k - Identifies if source registers are waiting on a result from another functional unit. FU

R_j, R_k - Flags that say if the operands (F_j, F_k) are ready (Yes/No).

Advantages	Disadvantages
Simpler hardware than Tomasulo's	No register renaming → more stalls
Centralized control makes tracking easier	Less efficient handling of WAR and WAW hazards
Supports out-of-order execution and completion	Central scoreboard can become a bottleneck
Handles RAW, WAR, and WAW hazards	Functional units stall if operands aren't ready
Easier to implement precise exceptions	Less scalable compared to distributed systems

Tomasulo's Algorithm is a smarter way to run instructions out of order. It was designed for the IBM 360/91, which had slow memory and very few registers. The goal was to improve performance even when compilers couldn't help.

Instead of tracking everything centrally (like Scoreboarding), Tomasulo distributed control to components called Reservation Stations.

Why Tomasulo Was Needed:

- The IBM 360 had only 4 floating-point registers, limiting optimization.

- Long memory delays needed better overlap and parallelism.
- Tomasulo's method let the hardware rename registers dynamically and avoid many stalls.

Key Concepts:

1. Reservation Stations (RS):

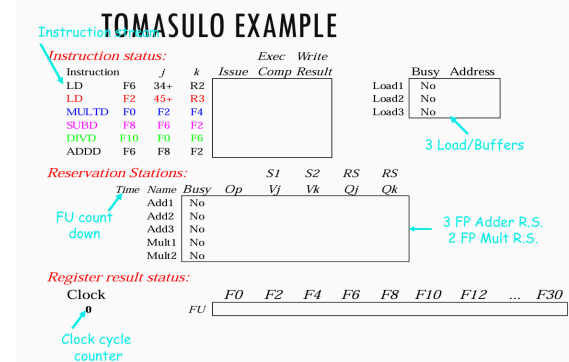
- Each functional unit has slots (RS) where instructions wait.
- Stores the operation, operands, and status.
- Tracks which instructions are waiting on what data.

2. Register Renaming:

- Avoids WAR and WAW hazards.
- Uses tags instead of actual register names to prevent conflicts.

3. Common Data Bus (CDB):

- Broadcasts results to all reservation stations.
- Instructions grab values from the bus as soon as they're available.



The 3 Stages of Tomasulo's Algorithm:

- Issue:**
 - If a reservation station is available, the instruction is issued.
 - Register operands are renamed: either loaded immediately or marked to wait.
- Execute:**
 - Runs when both operands are ready.
 - If not ready, the instruction listens to the CDB for the needed data.
- Write Result:**
 - When done, the result is broadcast on the CDB.
 - Any waiting instruction that needs it will grab the result.
 - The reservation station is then freed.

Operation	Cycles
Load	2
Add/Sub	2
Multiply	10
Divide	40

Reservation Station Components:

- **Op:** Operation to perform (+, -, *, /)
- **Vj, Vk:** Values of operands (if available)
- **Qj, Qk:** Tags indicating where the value will come from (if not ready) RS
- **Busy:** Whether the station is occupied
- **Register status:** Tracks which unit will write to which register

Benefits	Drawbacks
Distributed hazard detection via reservation stations	Hardware is more complex than scoreboarding
No need for a central scoreboard	Common Data Bus (CDB) can become a bottleneck
Register renaming eliminates WAR and WAW hazards	Only one instruction can broadcast on the CDB per cycle
Instructions issue, execute, and write independently (fully out-of-order)	Scaling to multiple CDBs requires extra logic and wiring

◆ Tomasulo's Algorithm	
Hazard	Handling Method
RAW	Stall till write back and start executing in that cycle only
WAW	Handled through overwrite in Register Status
WAR	Handled through register renaming (e.g., only accept read from L1 even if storing A6 at that dest)
◆ Scoreboarding	
Hazard	Handling Method
RAW	Stall till write back and start reading operands in next cycle
WAR	Stall till previous instruction reads operand and write back in next cycle
WAW	Cannot issue instruction until the previous one has written to that dest

Aspect	Superscalar Architecture	VLIW (Very Long Instruction Word) Architecture
Description	Executes multiple instructions per cycle using dynamic scheduling in hardware	Executes multiple instructions per cycle using static scheduling by compiler
Instruction Scheduling	Dynamic scheduling at runtime using current processor state and data dependencies	Static scheduling at compile time using compiler analysis
Scheduling Control	Controlled by hardware (instruction queues, renaming engines, reservation stations)	Controlled by compiler, which packs parallel instructions into a long word
Execution Order	Supports out-of-order execution for better resource utilization	Executes instructions in fixed parallel bundles in order as determined by compiler
Resource Allocation	Functional units assigned dynamically based on need and availability	Functional unit usage is explicitly encoded in instruction words
Parallelism Exposure	Implicit parallelism detected and managed by hardware	Explicit parallelism encoded in the VLIW instruction
Hardware Complexity	High complexity due to dynamic scheduling logic	Lower hardware complexity, relies on compiler for scheduling
Compiler Dependence	Less reliant on compiler for parallelism exploitation	Highly dependent on a smart compiler to detect and pack parallel instructions
Adaptability	High adaptability to various program structures at runtime	Limited adaptability; performance depends on compile-time decisions
Control Flow Handling	Can handle complex control flow (e.g., branches) via speculation and dynamic analysis	Requires special handling for branches and dependencies

Parallelism Potential	High, but extracting it is complex due to runtime checks	Limited by compiler ability and VLIW instruction format
Performance Consistency	Can adapt performance dynamically depending on code and input	Performs well only if compiler schedules instructions efficiently

Loop unrolling: Compiler optimization that replicates loop body multiple times to reduce iteration count, branch overhead, and expose instruction-level parallelism.

Performed by: Compilers (auto), programmers (manual).

Used in: High-performance computing, pipelined CPUs, Tomasulo's algorithm, to improve ILP, reduce hazards, enhance scheduling.

- It reduces the number of iterations by performing multiple operations per loop iteration.
- This minimizes the loop control overhead (like incrementing counters and checking conditions).
- It can expose more opportunities for instruction scheduling, parallelism, and reduced branching.

Original Loop:

```
c
for (int i = 0; i < 4; i++) {
    a[i] = a[i] + 1;
}
```

Unrolled Loop (by factor of 2):

```
c
for (int i = 0; i < 4; i += 2) {
    a[i] = a[i] + 1;
    a[i+1] = a[i+1] + 1;
}
```

- Now only **2 iterations** instead of 4.
- Each iteration does **2 updates** instead of 1.
- Branch and index update overhead is halved.

BEFORE MID 2 CONTENT

APPENDIX A - ISA

An **Instruction Set Architecture (ISA)** defines how a processor stores data, interacts with memory, and performs calculations. **Implicit Operand:** Automatically assumed by the CPU (e.g., `MUL AL`). **Explicit Operand:** Directly specified in the instruction (e.g., `ADD AX, BX`).

- In a **stack-based architecture**, implicit operands are stored in a stack, with operations using the top values, shrinking or growing the stack accordingly. It is simple and requires fewer instructions but is slow due to frequent push/pop operations. A **$C = A + B$** operation involves pushing A and B onto the stack, performing the addition, and storing the result in memory.
- An **accumulator-based architecture** uses a single special register, the accumulator, which stores intermediate results and is modified with each operation. It is simpler than stack-based architectures and reduces memory access but is inefficient for complex operations since the accumulator is constantly overwritten. For **$C = A + B$** , A is loaded into the accumulator, B is added, and the result is stored in memory. One of the operands is implicit. **Extended Accumulator Architecture:** Special-purpose registers with restricted use.
- **General-Purpose Register (GPR) Architecture: (all explicit operands):**
 - A **register-memory architecture** uses general-purpose registers while allowing direct memory access in calculations, making it more efficient than accumulator-based systems but still reliant on memory. In **$C = A + B$** , A is loaded from memory into a register, B is added directly from memory, and the result is stored.
 - A **register-register (load-store) architecture** performs all operations using registers, with memory access restricted to explicit load/store instructions. This approach is the fastest and most common today, as registers are significantly faster than memory. In **$C = A + B$** , A and B are first loaded into registers, the addition is performed, and the result is stored back in memory.
 - **Memory-Memory Architecture (obsolete):** Keeps all operands in memory.

Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1, A	Load R1, A
Push B	Add B	Add R3, R1, B	Load R2, B
Add	Store C	Store R3, C	Add R3, R1, R2
Pop C			Store R3, C

Figure A.2 The code sequence for $C = A + B$ for four classes of instruction sets. Note that the Add instruction has implicit operands for stack and accumulator architectures and explicit operands for register architectures. It is assumed that A, B, and C all belong in memory and that the values of A and B cannot be destroyed. [Figure A.1](#) shows the Add operation for each class of architecture.

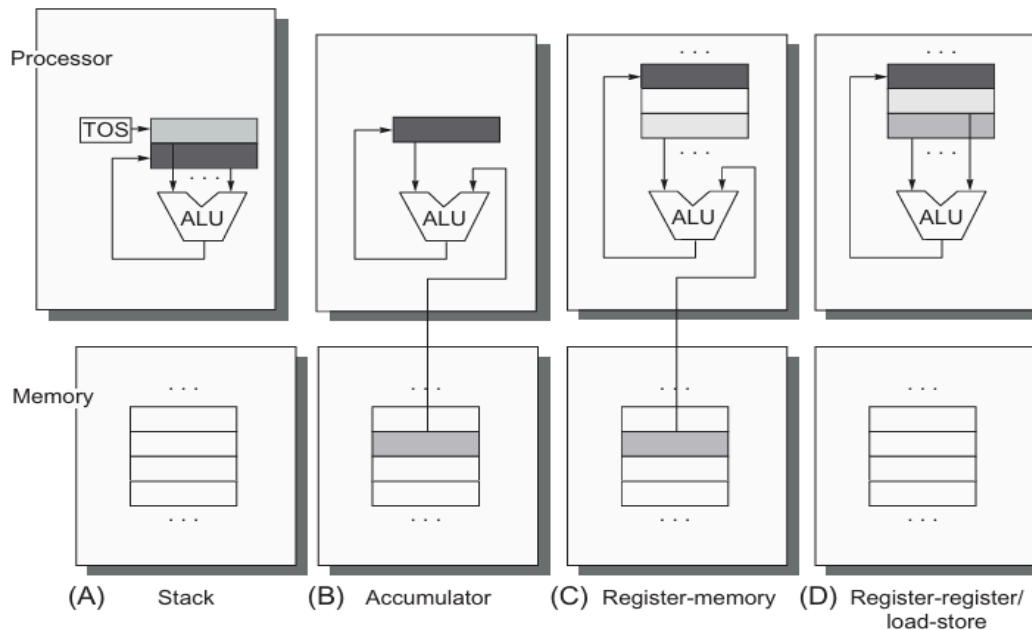


Figure A.1 Operand locations for four instruction set architecture classes. The arrows indicate whether the operand is an input or the result of the arithmetic-logical unit (ALU) operation, or both an input and result. Lighter shades indicate inputs, and the dark shade indicates the result. In (A), a top of stack (TOS) register points to the top input operand, which is combined with the operand below. The first operand is removed from the stack, the result takes the place of the second operand, and TOS is updated to point to the result. All operands are implicit. In (B), the accumulator is both an implicit input operand and a result. In (C), one input operand is a register, one is in memory, and the result goes to a register. All operands are registers in (D) and, like the stack architecture, can be transferred to memory only via separate instructions: push or pop for (A) and load or store for (D).

- The explicit operands may be accessed directly from memory or may need to be first loaded into temporary storage, depending on the class of architecture and choice of specific instruction.
- As the figures show, there are really two classes of register computers. One class can access memory as part of any instruction, called register-memory architecture, and the other can access memory only with load and store instructions, called load-store architecture. 3rd obsolete
- **Reasons for general-purpose registers (GPR):** faster than memory, more efficient for compilers, reduces memory traffic, and improves program speed and code density.
Compiler preferences: all registers should be general-purpose; larger register counts improve efficiency.
- The number of registers needed depends on compiler effectiveness, with modern compilers using more registers for expression evaluation, parameter passing, and variable storage, leading to higher register counts in recent architectures.
- **ISA Trade-offs:** impact on instruction encoding, number of instructions, and implementation strategies.
- **GPR architectures differ by two key traits:** whether ALU instructions use 2 operands (one acts as both source and destination) or 3 (separate source/destination), and how many operands can access memory (0 for load-store, 1 for register-memory, or 2+ for memory-memory)

Number of memory addresses	Maximum number of operands allowed	Type of architecture	Examples
0	3	Load-store	ARM, MIPS, PowerPC, SPARC, RISC-V
1	2	Register-memory	IBM 360/370, Intel 80x86, Motorola 68000, TI TMS320C54x
2	2	Memory-memory	VAX (also has three-operand formats)
3	3	Memory-memory	VAX (also has two-operand formats)

Figure A.3 Typical combinations of memory operands and total operands per typical ALU instruction with examples of computers. Computers with no memory reference per ALU instruction are called load-store or register-register computers. Instructions with multiple memory operands per typical ALU instruction are called register-memory or memory-memory, according to whether they have one or more than one memory operand.

Type	(Memory, Operands)	Advantages (Simple Terms)	Disadvantages (Simple Terms)
Register-register	(0, 3)	- Instructions are simple and fast - All work done using registers - Easy to manage	- You must load/store from memory separately - More instructions = bigger programs
Register-memory	(1, 2)	- Can use one memory operand directly - Shorter instructions overall	- One operand is overwritten - Mixing memory & register limits how many registers you can use
Memory-memory	(2 or 3, 2 or 3)	- Most compact - No need to use extra registers for temporary values	- Instructions vary a lot in size and speed - Slower due to many memory accesses (not used now)

Memory Addressing: Determines how memory addresses are interpreted and specified, impacting performance and compiler efficiency.

Address Interpretation:

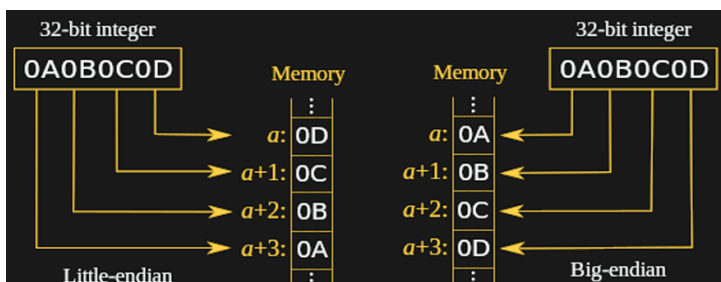
- **Byte-addressed systems** access bytes (8-bit), half-words (16-bit), words (32-bit), and double words (64-bit).
- **Byte Order:**

Little Endian: Least-significant byte stored at lowest address.

Big Endian: Most-significant byte stored at lowest address

Memory Alignment: refers to arranging data in memory at addresses that are multiples of their size, improving access speed and efficiency.

- **Aligned access:** Address $A \bmod s = 0$ for object size s .
- **Misaligned access:** Requires multiple memory references, reducing efficiency.
- **Example:** 4-byte word access aligned at **0, 4, 8...**, misaligned at **1, 2, 3...**



Memory address	Alignment (8 bit)	Alignment (16 bit)	Alignment (32 bit)	Alignment (64 bit)
0x0000_0000	Aligned	Aligned	Aligned	Aligned
0x0000_0001	Aligned	Non Aligned	Non Aligned	Non Aligned
0x0000_0002	Aligned	Aligned	Non Aligned	Non Aligned
0x0000_0003	Aligned	Non Aligned	Non Aligned	Non Aligned
0x0000_0004	Aligned	Aligned	Aligned	Non Aligned
0x0000_0005	Aligned	Non Aligned	Non Aligned	Non Aligned
0x0000_0006	Aligned	Aligned	Non Aligned	Non Aligned
0x0000_0007	Aligned	Non Aligned	Non Aligned	Non Aligned
0x0000_0008	Aligned	Aligned	Aligned	Aligned

Addressing Modes: define how an instruction specifies the actual effective address from the memory location/address of an operand using constants, registers, or memory locations.

Addressing Mode	Example Instruction	Meaning	When Used	Description
Direct (Absolute)	Add R1, (1001)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[1001]$	Accessing static data	Uses a fixed memory address specified in the instruction to retrieve data. Simple, lacks flexibility. Reduces computation time
Register & Memory Indirect	Add R4, (R1), Add R1, @(R3)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[\text{Regs}[R1]]$, $\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Mem}[\text{Regs}[R3]]]$ (double ptr)	Accessing using a pointer, computed address, or extra level of indirection	The memory address is stored in a register, which is dereferenced to get the value. Used to implement pointers, parameter passing, and linked data structures but may require multiple memory accesses.
Indexed (Base + Offset)	Add R3, (R1 + R2)	$\text{Regs}[R3] \leftarrow \text{Regs}[R3] + \text{Mem}[\text{Regs}[R1] + \text{Regs}[R2]]$	Useful in array addressing R2 can change during execution, then added to base	Uses a base register and an index register to dynamically calculate the memory address, useful for iterating through arrays.
Displacement	Add R4, (100 + R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[100 + \text{Regs}[R1]]$	Accessing local variables Fixed value added to base	Combines a base register with a fixed offset to locate data in memory. Commonly used for accessing local variables. Used for relocating a program, where the effective address (EA) is the sum of the base value and displacement.
Immediate	Add R4, 3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + 3$	For constants	Uses a fixed constant value directly in the instruction, avoiding memory access. Useful for adding small known values.
Register	Add R4, R3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Regs}[R3]$	When a value is in a register	The operands are stored in registers, making it the fastest mode as it does not access memory.
Autoincrement	Add R1, (R2)+	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]]$, then $\text{Regs}[R2] \leftarrow \text{Regs}[R2] + d$	Stepping through arrays in loops	Automatically increments the register after accessing memory, making it efficient for sequential data traversal like iterating through arrays.
Autodecrement	Add R1, -(R2)	$\text{Regs}[R2] \leftarrow \text{Regs}[R2] - d$, then $\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]]$	Similar to autoincrement, also for stack operations	Automatically decrements the register before accessing memory, commonly used in stack operations (push/pop) and reverse iteration.
Scaled	Add R1, 100(R2)[R3]	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[100 + \text{Regs}[R2] + \text{Regs}[R3] * d]$	Used in indexed array addressing	Similar to indexed addressing but includes a scaling factor (d), useful for accessing structured data like multi-dimensional arrays.

Addressing mode	Example instruction	Meaning	When used
Register	Add R4, R3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Regs}[R3]$	When a value is in a register
Immediate	Add R4, 3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + 3$	For constants
Displacement	Add R4, 100(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[100 + \text{Regs}[R1]]$	Accessing local variables (+ simulates register indirect, direct addressing modes)
Register indirect	Add R4, (R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[\text{Regs}[R1]]$	Accessing using a pointer or a computed address
Indexed	Add R3, (R1 + R2)	$\text{Regs}[R3] \leftarrow \text{Regs}[R3] + \text{Mem}[\text{Regs}[R1] + \text{Regs}[R2]]$	Sometimes useful in array addressing: R1 = base of array; R2 = index amount
Direct or absolute	Add R1, (1001)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[1001]$	Sometimes useful for accessing static data; address constant may need to be large
Memory indirect	Add R1, @(R3)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Mem}[\text{Regs}[R3]]]$	If R3 is the address of a pointer p , then mode yields $*p$
Autoincrement	Add R1, (R2)+	$\begin{aligned} \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \\ \text{Regs}[R2] &\leftarrow \text{Regs}[R2] + d \end{aligned}$	Useful for stepping through arrays within a loop. R2 points to start of array; each reference increments R2 by size of an element, d
Autodecrement	Add R1, -(R2)	$\begin{aligned} \text{Regs}[R2] &\leftarrow \text{Regs}[R2] - d \\ \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \end{aligned}$	Same use as autoincrement. Autodecrement/-increment can also act as push/pop to implement a stack.
Scaled	Add R1, 100(R2)[R3]	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[100 + \text{Regs}[R2] + \text{Regs}[R3] * d]$	Used to index arrays. May be applied to any indexed addressing mode in some computers

Addressing modes can significantly reduce instruction counts; they also add to the complexity of building a computer and may increase the average clock cycles per instruction (CPI) of computers that implement those modes. The **compiler** also determines how addressing modes are used and optimizes their selection to improve performance. Thus, the usage of various addressing modes is quite important in helping the architect choose what to include.

Operand types include character (8 bits), half-word (16 bits), word (32 bits), single-precision float (32 bits), double-precision float (64 bits), and integers (two's complement). Operand type is usually in the opcode; rarely, data is tagged for hardware interpretation. Control flow changes include conditional branches, jumps, procedure calls, and returns.

Instruction encoding styles include fixed-length (e.g., RISC, 32-bit), variable-length (e.g., x86, 1–17 bytes), and hybrid (e.g., RISC-V compressed). Compilers convert code from HLL to MC, manage instruction processing, and aim for accuracy, speed, and ease of debugging. Challenges include complexity, hardware-compiler mismatch, and limited optimization.

RISC-V Architecture

- RISC-V is a computer instruction set, which means it defines the basic commands a processor can understand and execute.
- A simple load-store instruction set designed for pipelining efficiency, with a fixed instruction set encoding, ensuring efficiency as a compiler target.
- RISC-V has three base instruction sets (**32-bit (RV32I)**, **64-bit (RV64I)**, and a future **128-bit (RV128I)** version) supporting 32-bit or 64-bit integers, with optional extensions for various functionalities like floating-point and vector operations. This flexibility allows RISC-V to be used in a wide range of applications, from low-cost embedded processors to high-end configurations.
- Processors can be simple (cheap and small) or complex (powerful and expensive) depending on which parts of the RISC-V set they use.

Basics of the RISC-V Instruction Set

RISC-V follows a load-store architecture, simplifying pipeline implementation with:

- Operations acting on registers only.
- Memory accessed only via load/store instructions.
- Fixed instruction formats for easy decoding.

Registers in RISC-V

- **RV64G (64-bit version) has 32 general-purpose registers (GPRs), named x0 to x31.**
- Some versions also include **32 floating-point registers (FPRs)** for handling decimal numbers. (can be **32-bit (single precision)** or **64-bit (double precision)**.)
- A full-featured RISC-V processor might be **RV64G**, which includes the **M, A, F, and D** extensions.

Name of base or extension	Functionality
RV32I	Base 32-bit integer instruction set with 32 registers
RV32E	Base 32-bit instruction set but with only 16 registers; intended for very low-end embedded applications
RV64I	Base 64-bit instruction set; all registers are 64-bits, and instructions to move 64-bit from/to the registers (LD and SD) are added
M	Adds integer multiply and divide instructions
A	Adds atomic instructions needed for concurrent processing; see Chapter 5
F	Adds single precision (32-bit) IEEE floating point, includes 32 32-bit floating point registers, instructions to load and store those registers and operate on them
D	Extends floating point to double precision, 64-bit, making the registers 64-bits, adding instructions to load, store, and operate on the registers
Q	Further extends floating point to add support for quad precision, adding 128-bit operations
L	Adds support for 64- and 128-bit decimal floating point for the IEEE standard
C	Defines a compressed version of the instruction set intended for small-memory-sized embedded applications. Defines 16-bit versions of common RV32I instructions
V	A future extension to support vector operations (see Chapter 4)
B	A future extension to support operations on bit fields
T	A future extension to support transactional memory
P	An extension to support packed SIMD instructions: see Chapter 4
RV128I	A future base instruction set providing a 128-bit address space

Register	Name	Use
x0	zero	The constant value 0
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5-x7	t0-t2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10-x11	a0-a1	Function arguments/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved registers
x28-x31	t3-t6	Temporaries
f0-f7	ft0-ft7	FP temporaries
f8-f9	fs0-fs1	FP saved registers
f10-f11	fa0-fa1	FP function arguments/return values
f12-f17	fa2-fa7	FP function arguments
f18-f27	fs2-fs11	FP saved registers
f28-f31	ft8-ft11	FP temporaries

Figure A.22 RISC-V has three base instructions sets (and a reserved spot for a future

Data Types in RISC-V

- Supports **8-bit (bytes)**, **16-bit (half-words)**, **32-bit (words)**, and **64-bit (double words)**.
- Floating-point numbers can be **32-bit (single precision)** or **64-bit (double precision)**.

Memory and Addressing

- The architecture supports two data addressing modes: **immediate** and **displacement**, with 12-bit fields. **Register indirect** is accomplished simply by placing 0 in the 12-bit displacement field, and **limited absolute addressing** with a 12-bit field is accomplished by using register 0 as the base register.
- RV64G is a byte-addressable, 64-bit Little Endian memory system, using load-store architecture for memory access.
- Memory accesses can involve GPRs and FPRs, with support for various data types, though unaligned accesses are discouraged due to potential slow performance.

Instruction Formats

- Instructions are 32-bits long with 7-bit primary opcode and categorized into:
 - **R-type** – (Register-Register Instructions): Used for arithmetic operations (like addition and multiplication) that involve two registers as inputs and store the result in another register.
 - **I-type** – (Immediate & Load Instructions): Used for operations that involve a register and an immediate (a fixed number), such as loading a value from memory or performing an arithmetic operation with a constant.
 - **S-type** – (Store Instructions): Used for storing data from a register into memory.
 - **U-type** – (Jump & Link Instructions): Used for jumping to a different part of a program (useful for function calls).

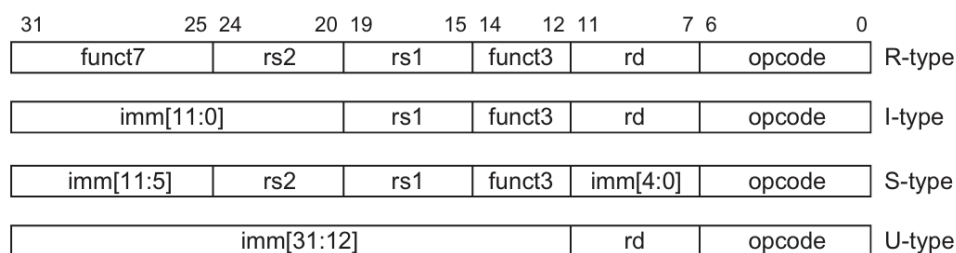


Figure A.23 The RISC-V instruction layout. There are two variations on these formats, called the SB and UJ formats; they deal with a slightly different treatment for immediate fields.

Instruction format	Primary use	rd	rs1	rs2	Immediate
R-type	Register-register ALU instructions	Destination	First source	Second source	
I-type	ALU immediates Load	Destination	First source base register		Value displacement
S-type	Store Compare and branch		Base register first source	Data source to store second source	Displacement offset
U-type	Jump and link Jump and link register	Register destination for return PC	Target address for jump and link register		Target address for jump and link

Figure A.24 The use of instruction fields for each instruction type. Primary use shows the major instructions that use the format. A blank indicates that the corresponding field is not present in this instruction type. The I-format is used for both loads and ALU immediates, with the 12-bit immediate holding either the value for an immediate or the displacement for a load. Similarly, the S-format encodes both store instructions (where the first source register is the base register and the second contains the register source for the value to store) and compare and branch instructions (where the register fields contain the sources to compare and the immediate field specifies the offset of the branch target). There are actually two other formats: SB and UJ that follow the same basic organization as S and J, but slightly modify the interpretation of the immediate fields.

Operations in RISC-V

1. RISC-V supports four broad classes of instructions: **loads and stores, ALU operations, branches and jumps, and floating-point operations.**
2. Memory operations include 8, 16, 32, and 64-bit loads and stores for integer and floating-point registers, with specific instructions for signed and unsigned operations.
3. Arithmetic and logical operations in RISC-V include simple add, subtract, AND, OR, shifts, and more, with both register-register and register-immediate formats.
4. Control flow is managed through conditional branches and jumps, with support for both PC-relative and register-relative addressing modes.
5. Floating-point operations follow the IEEE 754 format, supporting single and double precision with various arithmetic, comparison, and conversion instructions between integer and floating-point registers.

SEE PIPELINING BOOK PDF AND SLIDES

Metric	Formula
Total Cycles (Non-Pipelined)	$n \times k$
Total Cycles (Pipelined)	$n + k - 1$
Time per Instruction (Non-Pipelined)	$k \times \text{Clock Cycle Time}$
Time per Instruction (Pipelined)	Clock Cycle Time (ideal)
Time per Instr. (Ideal Pipelined)	$\text{Time (Unpipelined)} / k$
Total Exec Time (Non-Pipelined)	$n \times k \times \text{Clock Cycle Time}$
Total Exec Time (Pipelined)	$(n + k - 1) \times \text{Clock Cycle Time}$
Speedup from Pipelining	$\text{Unpipelined Time} / \text{Pipelined Time}$ OR $(n \times k) / (n + k - 1)$
Pipeline Utilization	$n / (n + k - 1)$ OR $\text{Slots Used} / \text{Total Slots}$
Clock Cycle Time	$\text{Total Time} / \text{Total Cycles}$ OR $\max(\text{Stage Delays}) + \text{Intermediate Register Delay}$
Pipelined CPI	$1 + \text{Stall cycles per instruction}$
Speedup (with stalls)	$\text{CPI}_{\text{unpipelined}} / (1 + \text{Stall cycles per instruction})$
Speedup (ideal)	$\text{Pipeline depth} / (1 + \text{Stall cycles per instruction})$

Speedup (Balanced Stages, CPI 1)	$1 / (1 + \text{Stall cycles per instruction})$
Pipeline Stall Cycles (Branches)	$\text{Branch Frequency} \times \text{Branch Penalty}$
Pipeline Speedup (Branch Penalties)	$k / (1 + \text{Branch Frequency} \times \text{Branch Penalty})$
Clock Period Calculation	$T = \max(\text{Stage Delays}) + d$
Pipeline Execution Time	$(k + n - 1) \times T$
Clock Frequency Calculation	$f = 1 / T$
Balancing Pipeline Stages	$T = \max(\text{Stage Delays}) + 1$ (to balance stages if unbalanced)
Pipeline Execution Cycles (Ideal)	$k + n - 1$ (no stalls/hazards)
Avg Time (Unpipelined)	$\text{CPI}_{\text{unpipelined}} \times \text{Clock_cycle_unpipelined}$
Avg Time (Pipelined)	$\text{CPI}_{\text{pipelined}} \times \text{Clock_cycle_pipelined}$
Initiation Interval	Time between starting two consecutive instructions (usually 1 in ideal pipeline)
Latency	Time from instruction entry to completion (= $k \times \text{Clock Cycle}$)
Non-pipelined Instruction (e.g., div)	High latency and high initiation interval

EXAMPLE QS:

Solution: Time Consumed in Processing 10,000 Instructions Using Pipeline

Given Data:

- Number of pipeline stages $s = 5$
- Stage delays (in nanoseconds):
 - Stage 1: 300 ns
 - Stage 2: 240 ns
 - Stage 3: 320 ns
 - Stage 4: 280 ns
 - Stage 5: 220 ns
- Intermediate register delay = 9 ns
- Number of instructions $n = 10,000$

Step 1: Determine Clock Cycle Time

The clock cycle time in a pipelined system is determined by the longest stage delay + intermediate register delay:

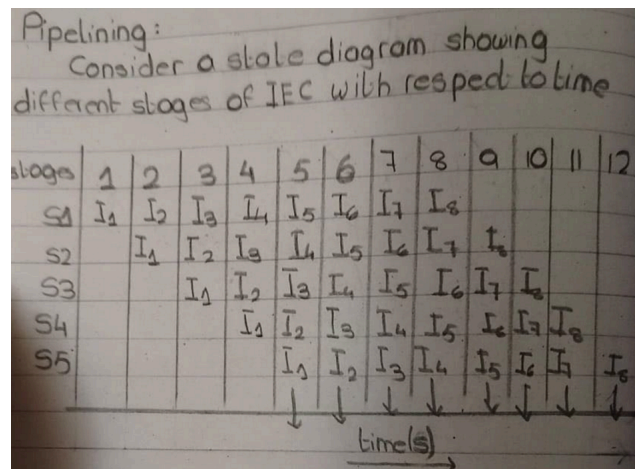
Clock Cycle Time = max(Stage Delays) + Intermediate Register Delay
Clock Cycle Time = max(300,240,320,280,220) + 9 = 320 + 9 = 329 ns

Step 2: Determine Total Execution Time in Pipeline

Total execution time for a pipeline is given by:

Total Execution Time = $(n + s - 1) \times \text{Clock Cycle Time}$

Total Execution Time = $(10,000 + 5 - 1) \times 329 = 10,004 \times 329 = 3,294,316 \text{ ns}$



1. Without Pipelining:

- Each instruction takes **5 cycles** (one per stage).
- 8 instructions take:** $8 \times 5 = 40 \text{ cycles}$

2. With Pipelining:

- The first instruction takes **5 cycles**.
- Each subsequent instruction enters the pipeline **one cycle after the previous one**.
- Total cycles needed for **8 instructions** in a 5-stage pipeline: $8 + 5 - 1 = 12 \text{ cycles}$

3. Speedup Calculation:

- Speedup** is the ratio of execution time without pipelining to execution time with pipelining: $40/12 \approx 3.33$
- This means pipelining makes execution approximately **3.33 times faster**.

4. Pipeline Utilization:

- Formula:** $\text{Slots Used/Total Slots} = 40/60 = 66\%$
- The **utilization of the pipeline is 66%**, meaning some pipeline slots remain unused due to stalls or dependencies.

Pipeline Hazards Overview

Pipeline hazards are issues that arise in pipelined CPU architecture and can be classified into:

1. **Data Hazards** – Occur when instructions depend on the results of previous instructions.
2. **Control Hazards** – Occur due to branch instructions affecting instruction flow.
3. **Structural Hazards** – Happen when hardware resources are insufficient.

Example 1: Data Hazard (RAW - Read After Write)

Given Instructions:

```
I1 → IF/D   OF   EX   Mem   WB
I2 →           IF/D   OF   EX   Mem   WB
```

```
I1: R3 ← R3 + R4   (Write to R3)
I2: R6 ← R3 + R5   (Read from R3)
```

Issue:

- Instruction I₂ reads R₃ before I₁ writes its updated value.
- This results in an incorrect computation.

Solution (Forwarding or Stalling):

- **Forwarding:** The output from I₁ (ALU result) can be directly fed into I₂ as an input.
 - **Stalling:** Introduce a delay (pipeline bubble) so that I₂ waits until I₁ completes.
-

Example 2: Control Hazard (Branch Instruction)

Pipeline Stages for Multiple Instructions:

```
I1: IF -> ID -> OF -> EX -> MEM -> WB
I2:      IF -> ID -> OF -> EX -> MEM -> WB
I3:      IF -> ID -> OF -> EX -> MEM -> WB
I4:      IF -> ID -> OF -> EX -> MEM -> WB
```

Issue:

- If I₁ is a conditional branch, the next instruction fetch (I₂, I₃, etc.) might be wrong.
- If the branch is taken, the pipeline must flush incorrect instructions.

Solution:

- **Branch Prediction:** Predict whether the branch is taken and execute accordingly.

- **Pipeline Flushing:** If the prediction is wrong, flush the pipeline and restart execution.
-

Example 3: Structural Hazard

Occurs when two instructions require the same hardware resource (e.g., memory access).

Solution:

- Use separate instruction and data memory (Harvard Architecture).
 - Introduce pipeline registers to avoid resource conflicts.
-

Remedies for Pipeline Hazards

1. **RAW (Read After Write)** → Forwarding/Circuit Switching.
 2. **WAR (Write After Read) & WAW (Write After Write)** → Register Renaming.
 3. **Stalls (if needed)** to allow dependencies to resolve.
-

Problem 1: Clock Time Period Calculation

Given:

- 5-stage pipeline with stage delays: 1, 2, 3, 1, 1 units
- Latch delay $d = 0$

Formula:

$$T = \max(\text{stage delays}) + d$$

Solution:

$$T = \max(1, 2, 3, 1, 1) + 0 = 3 \text{ units}$$

Answer: The clock time period is **3 units**.

Problem 2: Pipeline Execution Time for Two Processors

Given:

- **Processor P₁:** 5-stage pipeline with stage delays 3, 2, 4, 2, 3 ms
- **Processor P₂:** 8-stage pipeline with each stage taking 2 ms
- **n = 100** items processed

Formula for Execution Time:

$$\text{Time} = (k+n-1) \times T$$

where:

- k = number of stages
- T = clock period (max stage delay for P_1 , fixed for P_2)

For Processor P_1 :

$$T = \max(3, 2, 4, 2, 3) = 4 \text{ ms}$$
$$(5+100-1) \times 4 = 104 \times 4 = 416$$

For Processor P_2 :

$$T = 2 \text{ ms (fixed)} \quad T = 2 \text{ ms}$$
$$(8+100-1) \times 2 = 107 \times 2 = 214 \text{ ms}$$

Answer:

- P_1 Execution Time: **416 ms**
 - P_2 Execution Time: **214 ms**
-

Problem 3: Balanced Pipeline Clock Frequency

Given:

- 5-stage pipeline with delays: 10, 12, 13, 12, 14 ns
- Latch delay = 1 ns

Step 1: Compute Clock Period

$$T = \max(10, 12, 13, 12, 14) + 1 = 15 \text{ ns}$$

Step 2: Compute Clock Frequency

$$f = 1/T = 1/15 \times 10^{-9} = 6.67 \times 10^7$$

Step 3: Balancing Pipeline

To balance, the slowest stage determines T . Since **14 ns is the bottleneck**,

$$T = 14 + 1 = 15 \text{ ns (same as original)}$$

No further balancing is possible without modifying stage delays.

Answer:

Clock frequency = **66.67 MHz**

Clock period = **15 ns**

Problem 4: Pipeline Cycles Calculation

Given Instructions:

1. $R4 = R5 + R6$
2. $R2 = R2 + R0$
3. $R3 = R2 + R0$
4. $R4 = R5 - R6$
5. $R2 = 10m[R3 + 10]$
6. $R3 = 10m[R2 + 0]$
7. $R4 = R9 = R8$

Assumptions:

- 5-stage pipeline (IF, ID, EX, MEM, WB)
- No forwarding or hazards (ideal case)

Formula for Pipeline Cycles:

Total Cycles = $k + n - 1$

where:

- $k = 5$ (pipeline stages)
- $n = 7$ (instructions)

Solution:

Total Cycles = $5 + 7 - 1 = 11$ cycles

Answer:

Total cycles consumed = **11 cycles**

Key Takeaways:

1. **Clock Period (T)** = Slowest stage delay + latch delay.
2. **Pipeline Execution Time** = $(k + n - 1) \times T$ (k + n - 1) \times T

3. **Balancing Pipeline:** Slowest stage determines **T**; frequency is $1/T$.
 4. **Ideal Pipeline Cycles** = $k+n-1$ (assuming no stalls or hazards).
-

Given:

- Clock cycle = 0.5 ns
- ALU (40%) = 4 cycles
- Branch (20%) = 4 cycles
- Memory (40%) = 5 cycles
- Pipelining overhead = 0.1 ns

♦ **Unpipelined Avg Time:**

$$\begin{aligned} &= 0.5 \text{ ns} \times (0.4 \times 4 + 0.2 \times 4 + 0.4 \times 5) \\ &= 0.5 \text{ ns} \times (1.6 + 0.8 + 2.0) \\ &= 0.5 \text{ ns} \times 4.4 \\ &= 2.2 \text{ ns} \end{aligned}$$

♦ **Pipelined Time:**

$$= 0.5 \text{ ns} + 0.1 \text{ ns} = 0.6 \text{ ns}$$

♦ **Speedup:**

$$= 2.2 \text{ ns} / 0.6 \text{ ns} \approx 3.7\times$$

✓ **Pipelining makes it 3.7 times faster.**

Given:

- 4 pipeline stages: IF, ID, EX, WB
- All stages take **1 cycle**, except:
 - EX:
 - **ADD / SUB** takes **1 cycle**
 - **MUL** takes **3 cycles**
- **Operand forwarding is allowed** (so we don't always wait for WB stage)



Instructions:

I1: ADD x2, x1, x0 → EX = 1 cycle
I2: MUL x4, x3, x2 → EX = 3 cycles

I3: SUB x6, x5, x4 → EX = 1 cycle

Key Idea:

- Start a new instruction in each clock cycle (standard pipelining).
- But if a later instruction depends on a previous one **still in EX**, it might **stall**.
- With **forwarding**, we avoid some stalls, but **can't read from a register before it's written**.

Execution Chart (as drawn in the image):

Instr	1	2	3	4	5	6	7	8
I1	IF	ID	EX	WB				
I2		IF	ID	EX			WB	
I3			IF	ID			EX	WB

Final Answer:

✓ 8 clock cycles to complete all instructions.

Given:

- 4 phases: 60, 50, 90, 80 ms
- Latch delay = 10 ms
- Number of tasks = 1000

(a) Pipeline cycle time **tp**:

Max stage time + latch delay = 90 + 10 = **100 ms**

(b) Non-pipelined execution time **tnp**:

Sum of all stages = 60 + 50 + 90 + 80 = **280 ms**

(c) Speedup **S**:

$$S = t_{np} / t_p = 280 / 100 = 2.8$$

(d) Pipeline time for 1000 tasks **t_{pipeline}** :

$$\text{Formula: } (k + n - 1) \times t_p$$

$$= (4 + 1000 - 1) \times 100 = 1003 \times 100 = 100300 \text{ ms}$$

(e) Sequential time for 1000 tasks **t_{seq}** :

$$= 1000 \times t_{np} = 1000 \times 280 = 280000 \text{ ms}$$

(f) Throughput:

$$= 1000 / 100300 \approx 0.00997 \text{ tasks/ms}$$

Given:

- Instructions:
 - I0: $R1 = R1 + R0$
 - I1: $R2 = R3 - R1 \rightarrow$ depends on I0
 - I2: $R4 = R5 + R6$
 - I3: $R2 = \text{Mem}[R3 + 100]$
 - I4: $R2 = \text{Mem}[R2 + 0] \rightarrow$ depends on I3
 - I5: $R2 = R2 \& R1 \rightarrow$ depends on I4
 - I6: $\text{if } (R2 == R1) \text{ goto B} \rightarrow$ depends on I5
 - I7: $T5 = \text{Mem}[R2 + 100] \rightarrow$ depends on I5
- Operand forwarding is **enabled**
- Hazard-handling introduces **stalls where needed**

Solution Table:

Instr	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
I0	F	D	E	M	WB												
I1		F	F	D	E	M	WB										
I2				F	D	E	M	WB									
I3					F	D	E	M	WB								
I4						F	F	D	E	M	WB						

I5								F	F	D	E	M	WB				
I6										F	D	E	M	WB			
I7											F	D	E	M	WB		

Answer: Total clock cycles = 15

Given Frequencies:

- Unconditional branch: 4% = 0.04
- Conditional branch (untaken): 6% = 0.06
- Conditional branch (taken): 10% = 0.10

Branch Penalties per Scheme (from Figure C.12 and C.13):

Scheme	Uncond (×0.04)	Untaken (×0.06)	Taken (×0.10)	Total CPI Addition
Stall pipeline	2×0.04 = 0.08	3×0.06 = 0.18	3×0.10 = 0.30	0.56
Predicted taken	2×0.04 = 0.08	3×0.06 = 0.18	2×0.10 = 0.20	0.46
Predicted untaken	2×0.04 = 0.08	0×0.06 = 0.00	3×0.10 = 0.30	0.38

So, if base CPI = 1:

- Stall scheme: Total CPI = 1.56
- Predicted taken: CPI = 1.46
- Predicted untaken: CPI = 1.38

✓ Speedup examples:

- Predicted untaken vs Stall: $1.56 / 1.38 \approx 1.13\times$
- Ideal (no stalls) vs Stall: $1.56 / 1.00 = 1.56\times$

CHAPTER 1 NUMERICAL FORMULAS

Bandwidth (maximum data transfer rate of a network) → **Formula:**

$$\text{Bandwidth} = \frac{\text{Total Data Capacity}}{\text{Time}}$$

Throughput (actual data transferred over a network in a given time) → **Formula:**

$$\text{Throughput} = \frac{\text{Total Data Transferred}}{\text{Time}}$$

Total Latency Formula:

$$\text{Latency} = \text{Propagation Time} + \text{Transmission Time} + \text{Queuing Time}$$

Dynamic Power: Power consumed during circuit switching (dependent on voltage, frequency, and capacitance).

Static Power: Power consumed when the circuit is idle due to leakage currents. Static power (due to leakage current when transistors are off) is becoming a significant concern.

$$P_{\text{dynamic}} = \frac{1}{2}CV^2f \quad E_{\text{dynamic}} = \frac{1}{2}CV^2$$
$$P_{\text{static}} = I_{\text{leakage}}V \quad E_{\text{static}} = P_{\text{static}} \times t$$

Failure Rate & Mean Time to Failure (MTTF):

$$\lambda_{\text{system}} = \lambda_{\text{component}} \times n$$

$$\text{MTTF} = \frac{1}{\lambda}$$

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

Failure Rate in FIT:

$$\lambda(\text{FIT}) = \lambda(\text{failures per hour}) \times 10^9$$

Availability:

$$A_{\text{system}} = (A_{\text{component}})^n$$

$$A = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

Redundant Power Supply MTTF Approximation:

$$\text{MTTF}_{\text{redundant}} = \frac{\text{MTTF}^2}{2 \times \text{MTTR}}$$

Concept	Formula/Definition
Speedup Definition	$\text{Speedup} = \frac{\text{Performance with enhancement}}{\text{Performance without enhancement}}$
Alternative Speedup Formula	$\text{Speedup} = \frac{\text{Execution Time without enhancement}}{\text{Execution Time with enhancement}}$
Fraction of Computation Time Enhanced	$\text{Fraction}_{\text{enhanced}} = \frac{\text{Time enhanced}}{\text{Total execution time}}$
Speedup from Enhanced Mode	$\text{Speedup}_{\text{enhanced}} = \frac{\text{Original execution time}}{\text{Enhanced execution time}}$
New Execution Time Formula	$\text{Execution Time}_{\text{new}} = \text{Execution Time}_{\text{old}} \times \left((1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} \right)$
Overall Speedup Formula	$\text{Speedup}_{\text{overall}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$

Amdahl's Law formula:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

Given:

- P (Fraction of execution time improved) = 40% = 0.40
- N (Speedup factor) = 4

$$\text{Fraction}_{\text{enhanced}} = \frac{\text{Time affected by enhancement}}{\text{Total execution time}}$$

$$\text{Speedup}_{\text{enhanced}} = \frac{\text{Execution time of original mode}}{\text{Execution time of enhanced mode}}$$

CPU Time:

$$\text{CPU Time} = \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}$$

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{Clock Cycle Time}$$

Clock Cycles:

$$\text{CPU Clock Cycles} = \sum_{i=1}^n (\text{IC}_i \times \text{CPI}_i)$$

CPI Calculation:

$$\text{CPI} = \frac{\sum_{i=1}^n (\text{IC}_i \times \text{CPI}_i)}{\text{Instruction Count}}$$

or

$$\text{CPI} = \sum_{i=1}^n \left(\frac{\text{IC}_i}{\text{Instruction Count}} \times \text{CPI}_i \right)$$

Clock Rate:

$$\text{Clock Rate} = \frac{1}{\underset{\downarrow}{\text{Clock Cycle Time}}}$$

Metric	Formula	Description
Total CPU Clock Cycles	$\sum_{i=1}^n IC_i \times CPI_i$	Total number of clock cycles required to execute a program.
CPU Execution Time	$(\sum_{i=1}^n IC_i \times CPI_i) \times \text{Clock cycle time}$	Total time taken by the CPU to execute the program.
Overall CPI	$\frac{\sum_{i=1}^n IC_i \times CPI_i}{\text{Instruction count}}$	Average number of cycles per instruction across all instructions.
Alternative CPI Representation	$\sum_{i=1}^n \left(\frac{IC_i}{\text{Instruction count}} \times CPI_i \right)$	Weighted sum of individual CPIs based on instruction frequency.
CPU Time (Execution Time)	$T = CPI \times IC \times t$ or $T = \frac{CPI \times IC}{f}$	t = clock cycle time, f = clock frequency.
CPI (Cycles Per Instruction)	$CPI = \frac{\text{Total Cycles}}{\text{Instruction Count}}$	Average number of cycles per instruction.
Instruction Count (IC)	Total number of instructions executed in a program.	Measures the program's size in terms of executed instructions.

Formula	Description
CPU Time (Execution Time)	$T = CPI \times IC \times t$ or $T = \frac{CPI \times IC}{f}$, where t = clock cycle time and f = clock frequency.
Execution Time with Memory Access Delays	$T = IC \times CP + M \times K \times t$, where CP = clock cycles per instruction, M = memory accesses, K = memory stall cycles.
Throughput	$\text{Throughput} = \frac{MIPS \times 10^6}{IC}$
Work Done per Unit Time (W)	$W = \frac{1}{T}$
Clock Cycle Time	$\text{Clock Cycle Time} = \frac{1}{\text{Clock Rate}}$
CPI (Cycles Per Instruction)	$CPI = \frac{\text{Total Cycles}}{\text{Instruction Count}}$
Instruction Count (IC)	Total number of instructions executed in a program.
MIPS (Millions of Instructions Per Second)	$MIPS = \frac{\text{Instruction Count}}{\text{Execution Time} \times 10^6}$

$$MIPS = \frac{IC}{T \times 10^6}$$

$$MIPS = \frac{f}{CPI \times 10^6}$$

$$MIPS = \frac{f \times IC}{C \times 10^6}$$

$$T = \frac{CPI \times IC}{f}$$

$$T = \frac{IC}{MIPS \times 10^6}$$

Average Memory Access Time (AMAT)	AMAT = Hit Time + (Miss Rate × Miss Penalty)	
Misses per Instruction	Misses per Instruction = Miss Rate × Memory Accesses per Instruction	
Cache Index Size	$2^{\text{index}} = \frac{\text{Cache Size}}{\text{Block Size} \times \text{Set Associativity}}$	
Power Utilization Effectiveness (PUE) in Datacenters	$\text{PUE} = \frac{\text{Total Facility Power}}{\text{IT Equipment Power}}$	
Performance Comparison (Relative Performance)	$\frac{\text{Execution Time}_Y}{\text{Execution Time}_X} = \frac{\text{Performance}_X}{\text{Performance}_Y}$	
Electrical Resistance	$R = \rho \frac{L}{A}$ (Resistivity × Length / Cross-sectional Area)	
Latency (Time Delay in Transmission)	Latency = $\frac{\text{Distance}}{\text{Propagation Speed}}$ + Transmission Time + Queueing Time	
Propagation Time	Propagation Time = $\frac{\text{Distance}}{\text{Propagation Speed}}$	
Transmission Time	Transmission Time = $\frac{\text{Message Size}}{\text{Bandwidth}}$	
Speedup	Speedup = $\frac{\text{Execution Time of old}}{\text{Execution Time of new}}$	
Amdahl's Law	Speedup _{overall} = $\frac{1}{(1-f) + \frac{f}{\text{Speedup}_{\text{enhanced}}}}$, where f is the fraction of execution time affected by the enhancement.	
Availability	Availability = $\frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$ (Mean Time to Failure and Mean Time to Repair)	
TDP (Thermal Design Power)	Maximum sustained power consumption for thermal design.	
Dynamic Energy	$E_{\text{dynamic}} = \frac{1}{2} CV^2$ (Capacitive load × Voltage ²)	
Dynamic Power	$P_{\text{dynamic}} = \frac{1}{2} CV^2 f$ (Capacitive load × Voltage ² × Frequency)	
Static Power	$P_{\text{static}} = I_{\text{static}} V$ (Static Current × Voltage)	
Die Yield	Die Yield = $\frac{\text{Wafer Yield}}{(1 + \text{Defects per unit area} \times \text{Die Area})^N}$, where N is a process complexity factor.	
Cost of Integrated Circuit	$\frac{\text{Cost of die} + \text{Cost of testing die} + \text{Cost of packaging and final test}}{\text{Final test yield}}$	
Cost of Die	$\frac{\text{Cost of wafer}}{\text{Dies per wafer} \times \text{Die yield}}$	
Dies per Wafer	$\frac{\pi \times (\text{Wafer diameter}/2)^2}{\text{Die area}} - \frac{\pi \times \text{Wafer diameter}}{\sqrt{2} \times \text{Die area}}$	
Die Yield (Bose-Einstein Formula)	Wafer yield × $\frac{1}{(1 + \text{Defects per unit area} \times \text{Die area})^N}$	
Defects per Unit Area	0.016 – 0.057 defects per square cm (2010)	
Process-Complexity Factor (N)	11.5 – 15.5 (40 nm, 2010)	
Execution Time Ratio	$\frac{\text{Execution Time}_Y}{\text{Execution Time}_X} = n$	
Execution Time and Performance Relationship	Execution Time = $\frac{1}{\text{Performance}}$	

BOOK EXAMPLE NUMERICALS

Example Some microprocessors today are designed to have adjustable voltage, so a 15% reduction in voltage may result in a 15% reduction in frequency. What would be the impact on dynamic energy and on dynamic power?

Answer Because the capacitance is unchanged, the answer for energy is the ratio of the voltages

$$\frac{\text{Energy}_{\text{new}}}{\text{Energy}_{\text{old}}} = \frac{(\text{Voltage} \times 0.85)^2}{\text{Voltage}^2} = 0.85^2 = 0.72$$

which reduces energy to about 72% of the original. For power, we add the ratio of the frequencies

$$\frac{\text{Power}_{\text{new}}}{\text{Power}_{\text{old}}} = 0.72 \times \frac{(\text{Frequency switched} \times 0.85)}{\text{Frequency switched}} = 0.61$$

shrinking power to about 61% of the original.

Example Find the number of dies per 300 mm (30 cm) wafer for a die that is 1.5 cm on a side and for a die that is 1.0 cm on a side.

Answer When die area is 2.25 cm²:

$$\text{Dies per wafer} = \frac{\pi \times (30/2)^2}{2.25} - \frac{\pi \times 30}{\sqrt{2} \times 2.25} = \frac{706.9}{2.25} - \frac{94.2}{2.12} = 270$$

Because the area of the larger die is 2.25 times bigger, there are roughly 2.25 as many smaller dies per wafer:

$$\text{Dies per wafer} = \frac{\pi \times (30/2)^2}{1.00} - \frac{\pi \times 30}{\sqrt{2} \times 1.00} = \frac{706.9}{1.00} - \frac{94.2}{1.41} = 640$$

Example Find the die yield for dies that are 1.5 cm on a side and 1.0 cm on a side, assuming a defect density of 0.047 per cm² and N is 12.

Answer The total die areas are 2.25 and 1.00 cm². For the larger die, the yield is

$$\text{Die yield} = 1 / (1 + 0.047 \times 2.25)^{12} \times 270 = 120$$

For the smaller die, the yield is

$$\text{Die yield} = 1 / (1 + 0.047 \times 1.00)^{12} \times 640 = 444$$

The bottom line is the number of good dies per wafer. Less than half of all the large dies are good, but nearly 70% of the small dies are good.

Example Assume a disk subsystem with the following components and MTTF:

- 10 disks, each rated at 1,000,000-hour MTTF
- 1 ATA controller, 500,000-hour MTTF
- 1 power supply, 200,000-hour MTTF
- 1 fan, 200,000-hour MTTF
- 1 ATA cable, 1,000,000-hour MTTF

Using the simplifying assumptions that the lifetimes are exponentially distributed and that failures are independent, compute the MTTF of the system as a whole.

Answer The sum of the failure rates is

$$\begin{aligned}\text{Failure rate}_{\text{system}} &= 10 \times \frac{1}{1,000,000} + \frac{1}{500,000} + \frac{1}{200,000} + \frac{1}{200,000} + \frac{1}{1,000,000} \\ &= \frac{10 + 2 + 5 + 5 + 1}{1,000,000 \text{ hours}} = \frac{23}{1,000,000} = \frac{23,000}{1,000,000,000 \text{ hours}}\end{aligned}$$

or 23,000 FIT. The MTTF for the system is just the inverse of the failure rate

$$\text{MTTF}_{\text{system}} = \frac{1}{\text{Failure rate}_{\text{system}}} = \frac{1,000,000,000 \text{ hours}}{23,000} = 43,500 \text{ hours}$$

or just under 5 years.

Example Suppose that we want to enhance the processor used for web serving. The new processor is 10 times faster on computation in the web serving application than the old processor. Assuming that the original processor is busy with computation 40% of the time and is waiting for I/O 60% of the time, what is the overall speedup gained by incorporating the enhancement?

Answer $\text{Fraction}_{\text{enhanced}} = 0.4$; $\text{Speedup}_{\text{enhanced}} = 10$; $\text{Speedup}_{\text{overall}} = \frac{1}{0.6 + \frac{0.4}{10}} = \frac{1}{0.64} \approx 1.56$

Example A common transformation required in graphics processors is square root. Implementations of floating-point (FP) square root vary significantly in performance, especially among processors designed for graphics. Suppose FP square root (FSQRT) is responsible for 20% of the execution time of a critical graphics benchmark. One proposal is to enhance the FSQRT hardware and speed up this operation by a factor of 10. The other alternative is just to try to make all FP instructions in the graphics processor run faster by a factor of 1.6; FP instructions are responsible for half of the execution time for the application. The design team believes that they can make all FP instructions run 1.6 times faster with the same effort as required for the fast square root. Compare these two design alternatives.

Answer We can compare these two alternatives by comparing the speedups:

$$\text{Speedup}_{\text{FSQRT}} = \frac{1}{(1 - 0.2) + \frac{0.2}{10}} = \frac{1}{0.82} = 1.22$$

$$\text{Speedup}_{\text{FP}} = \frac{1}{(1 - 0.5) + \frac{0.5}{1.6}} = \frac{1}{0.8125} = 1.23$$

Improving the performance of the FP operations overall is slightly better because of the higher frequency.

I

Example The calculation of the failure rates of the disk subsystem was

$$\begin{aligned}\text{Failure rate}_{\text{system}} &= 10 \times \frac{1}{1,000,000} + \frac{1}{500,000} + \frac{1}{200,000} + \frac{1}{200,000} + \frac{1}{1,000,000} \\ &= \frac{10 + 2 + 5 + 5 + 1}{1,000,000 \text{ hours}} = \frac{23}{1,000,000 \text{ hours}}\end{aligned}$$

ter One *Fundamentals of Quantitative Design and Analysis*

Therefore the fraction of the failure rate that could be improved is 5 per million hours out of 23 for the whole system, or 0.22.

Answer The reliability improvement would be

$$\text{Improvement}_{\text{power supply pair}} = \frac{1}{(1 - 0.22) + \frac{0.22}{4150}} = \frac{1}{0.78} = 1.28$$

Despite an impressive $4150 \times$ improvement in reliability of one module, from the system's perspective, the change has a measurable but small benefit.

Example Disk subsystems often have redundant power supplies to improve dependability. Using the preceding components and MTTFs, calculate the reliability of redundant power supplies. Assume that one power supply is sufficient to run the disk subsystem and that we are adding one redundant power supply.

Answer We need a formula to show what to expect when we can tolerate a failure and still provide service. To simplify the calculations, we assume that the lifetimes of the components are exponentially distributed and that there is no dependency between the component failures. MTTF for our redundant power supplies is the mean time until one power supply fails divided by the chance that the other will fail before the first one is replaced. Thus, if the chance of a second failure before repair is small, then the MTTF of the pair is large.

Since we have two power supplies and independent failures, the mean time until one supply fails is $MTTF_{\text{power supply}}/2$. A good approximation of the probability of a second failure is MTTR over the mean time until the other power supply fails. Therefore a reasonable approximation for a redundant pair of power supplies is

$$MTTF_{\text{power supply pair}} = \frac{MTTF_{\text{power supply}}/2}{\frac{MTTR_{\text{power supply}}}{MTTF_{\text{power supply}}}} = \frac{MTTF_{\text{power supply}}^2/2}{MTTR_{\text{power supply}}} = \frac{MTTF_{\text{power supply}}^2}{2 \times MTTR_{\text{power supply}}}$$

Using the preceding MTTF numbers, if we assume it takes on average 24 hours for a human operator to notice that a power supply has failed and to replace it, the reliability of the fault tolerant pair of power supplies is

$$MTTF_{\text{power supply pair}} = \frac{MTTF_{\text{power supply}}^2}{2 \times MTTR_{\text{power supply}}} = \frac{200,000^2}{2 \times 24} \cong 830,000,000$$

making the pair about 4150 times more reliable than a single power supply.

Example Suppose we made the following measurements:

Frequency of FP operations = 25%

Average CPI of FP operations = 4.0

Average CPI of other instructions = 1.33

Frequency of FSQRT = 2%

CPI of FSQRT = 20

Assume that the two design alternatives are to decrease the CPI of FSQRT to 2 or to decrease the average CPI of all FP operations to 2.5. Compare these two design alternatives using the processor performance equation.

Answer First, observe that only the CPI changes; the clock rate and instruction count remain identical. We start by finding the original CPI with neither enhancement:

$$\begin{aligned} \text{CPI}_{\text{original}} &= \sum_{i=1}^n \text{CPI}_i \times \left(\frac{\text{IC}_i}{\text{Instruction count}} \right) \\ &= (4 \times 25\%) + (1.33 \times 75\%) = 2.0 \end{aligned}$$

We can compute the CPI for the enhanced FSQRT by subtracting the cycles saved from the original CPI:

$$\begin{aligned} \text{CPI}_{\text{with new FPSQR}} &= \text{CPI}_{\text{original}} - 2\% \times (\text{CPI}_{\text{old FPSQR}} - \text{CPI}_{\text{of new FPSQR only}}) \\ &= 2.0 - 2\% \times (20 - 2) = 1.64 \end{aligned}$$

We can compute the CPI for the enhancement of all FP instructions the same way or by summing the FP and non-FP CPIs. Using the latter gives us

$$\text{CPI}_{\text{new FP}} = (75\% \times 1.33) + (25\% \times 2.5) = 1.625$$

Since the CPI of the overall FP enhancement is slightly lower, its performance will be marginally better. Specifically, the speedup for the overall FP enhancement is

$$\begin{aligned} \text{Speedup}_{\text{new FP}} &= \frac{\text{CPU time}_{\text{original}}}{\text{CPU time}_{\text{new FP}}} = \frac{\text{IC} \times \text{Clock cycle} \times \text{CPI}_{\text{original}}}{\text{IC} \times \text{Clock cycle} \times \text{CPI}_{\text{new FP}}} \\ &= \frac{\text{CPI}_{\text{original}}}{\text{CPI}_{\text{new FP}}} = \frac{2.00}{1.625} = 1.23 \end{aligned}$$

Happily, we obtained this same speedup using Amdahl's Law on page 51.

MORE CLASS EXAMPLE QS IN NOTES